

SVMים לא-לינאריים: kernels

לפעמים קלאסיפיקציה לינארית לא מספיקה, כי הנקודות לא מתחלקות לפי קו ישר - למרות שיכול להיות שעדיין יש איזו צורה גיאומטרית. אבל אולי אם נוסיף מימד, כאשר הקואורדינטה על המימד החדש היא פונקציה (לא לינארית) של שאר הממדים, נוכל לקבל הפרדה לינארית.

לא חייבים ממש להוסיף מימד - אפשר פשוט לעשות טרנספורמציה לא-לינארית באותו מימד.

הגדרה: יש לנו נקודות אימון במרחב $\mathbb{R}^d$, אנחנו עושים להן טרנספורמציה לfeature space $\mathbb{R}^D$ שבו אפשר למצוא הפרדה לינארית.

נסמן את המיפוי $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$. בד"כ $D > d$ (מרחיבים feature)

בעיות: • הבעיה החדשה היא high bias problem.

– אבל אפשר לפתור את זה באמצעות רגולריזציה ובאמצעות הוספת נתונים.

• עלות חישובית של עבודה במימד יותר גבוה.

בשביל להתמודד עם זה, נשתמש בטריק עם SVM. כשעושים מינימיזציה ל-SVM לא תלויים בערכי ה-x, אלא רק במכפלות $\mathbf{x}_i^T \mathbf{x}_j$.

הטריק - רוצים לחשב את $\phi(\mathbf{x})^T \phi(\mathbf{z})$ בלי לחשב את $\phi(\mathbf{x})$ ו $\phi(\mathbf{z})$ ב $\mathcal{O}(d)$. בשביל זה משתמשים בפונקציית kernel:

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T \phi(\mathbf{z})$$

דוגמה: במרחב דו-מימדי עם $\phi(\mathbf{x}) = \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}$ הקרנל הוא

$$\phi(\mathbf{x})^T \phi(\mathbf{z}) = \dots = (\mathbf{x}^T \mathbf{z})^2$$

נשים לב: לא תמיד זה עוזר לנו. לפעמים עלות החישוב של $K(\mathbf{x}, \mathbf{z})$ היא עדיין $\mathcal{O}(D)$.

קרנל פולינומי

אפשר להכליל את הקרנל הפולינומי מדרגה p :

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + 1)^p$$

שכולל את כל המרכיבים:

$$(1, x_i, z_i, x_i z_j, x_i^2, z_i^2, x_i z_j^2, x_i^2 z_j, \dots, x_i^p z_j^p, \dots)$$

המימד של feature space הוא $D = \binom{d+p}{d}$. אם, למשל, $d = 100$ ו $p = 5$ אז המימד של המרחב הוא 96 מיליון!

The Radial Basis Function (RBF) Kernel

ה-RBF - ידוע גם בתור קרנל גאוסיאני - הוא הקרנל הכי נפוץ ב-SVM, ומוגדר בתור:

$$K(\mathbf{x}, \mathbf{z}) = e^{-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}}$$

- אינטואיציה: אם $\mathbf{x}$ מאוד קרוב ל- $\mathbf{z}$, אז $\|\mathbf{x} - \mathbf{z}\| \approx 0$ ולכן $K(\mathbf{x}, \mathbf{z}) \approx 1$.
- באופן דומה, אם $\mathbf{x}$ ו- $\mathbf{z}$ מאוד רחוקים, $K(\mathbf{x}, \mathbf{z}) \approx 0$.
- כלומר הקרנל הזה, במקום לחצות את המרחב עם hyperplane, מחבר ביחד נקודות קרובות.

ה-RBF מקביל ל- ϕ שממפה למרחב פיצ'רים אינסופי:

$$\phi(\mathbf{x}) = e^{-\frac{\|\mathbf{x}\|^2}{2\sigma^2}} \left(1, \frac{x_1}{\sigma}, \frac{x_2}{\sigma}, \dots, \frac{x_i x_j}{\sqrt{2}\sigma^2}, \dots \right)$$

מקבלים כאן מכפלה אינסופית, אבל היא טור טיילור שמתכנס לקרנל הסופי.

מה הופך קרנל לואלידי?

אי אפשר לבחור סתם כל טרנספורמציה שאנחנו רוצים. הקרנל חייב לייצג מכפלה פנימית $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T \phi(\mathbf{z})$.

נשים לב: $\phi(\mathbf{x})$ יכול למפות למימד אינסופי (למשל ב-RBF) אבל עדיין מייצג מכפלה פנימית.

לכל אוסף סופי של n נקודות אימון, מטריצת גראם היא

$$G_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$$

תנאי Mercer: קרנל הוא וואלידי אם"ם מטריצת גראם שלו הוא positive semi-definite.

חזרה ל-SVM

למרות שזה כבר לא לינארי, SVM עדיין ינסה למצוא margin.

עצי החלטה

באמצעות חלוקת המרחב למלבנים אפשר להתאים היפותזה לכל מדגם¹. נייצג את זה באמצעות עצי החלטה - וככה נוכל גם להתחשב במימדים שהם לא באמת מספרים.

אימון מודל מבוסס עצי החלטה

אנחנו רוצים למצוא עץ T שממזער את השגיאה:

$$\min_T Loss(T, D)$$

- בעיה:
- עבור d פיצ'רים בולאנים, יש 2^{2^d} פונקציות בוליאניות אפשריות - ומספר עצי ההחלטה האפשריים אפילו יותר גדול.
 - עבור $X \in \mathbb{R}^{n \times d}$, מרחב ההיפותזות האפקטיבי הוא בערך $(dn)^n$. המרחב הכולל הוא אמנם אינסופי, אבל ההחלטות שלנו הן לפי n הנתונים.
- זאת בעיה NP-קשה

הפתרון: חיפוש יוריסטי. לא נמצא את העץ הכי טוב

- הגישה:
- greedy ו top-down - נבנה את העץ מלמעלה למטה, וכל פעם נבחר את split הכי טוב לאותו שלב.
 - החלוקה לא חייבת להיות ל2. אם למשל יש פיצ'ר איכותני עם יותר מ2 אפשרויות.
 - רקורסיבי - מחלקים את הנתונים בכל שלב, ואז נכנסים לכל אחד מהחלקים ומחלקים שוב.
 - ממשיכים עד שמקבלים מלבנים הומוגניים.
 - אם אי אפשר לחלק (למשל כי אין פיצ'ר) אזור לא הומוגני, אז אין ברירה - מתייגים אותו עם התווית הכי נפוצה (בתוכו)

¹אלא אם כן יש $x_i = x_j$
 $y_i \neq y_j$

איך בוחרים חלוקה? באמצעות אנטרופיה:

- אנטרופיה נמוכה אומרת שהקבוצה היא "טהורה", שאפשר לומר עליה דברים בביטחון.
- אנטרופיה גבוהה אומרת שהקבוצה היא "מבולגנת", שיש ערבוב של תוויות.

עבור קבוצה S שכוללת דוגמאות חיוביות ושליליות, האנטרופיה מוגדרת כ:

$$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

או - עבור multiclass:

$$H(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

(p_i היא ההסתברות לקבל את class i בתוך S מתוך c class שונים)
האנטרופיה המקסימלית היא $\max H = \log_2 c$.
כדי לבחור חלוקה, נרצה לחשב את Information Gain שלה:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

כאשר A הוא המאפיין לפיו מחלקים ו- S_v היא תת הקבוצה של S בה $A = v$.
כדי שה-IG יהי הכי גדול, נרצה שהרכיב של הממוצע המשוקלל יהיה קטן.

אלגוריתם CART (Classification and Regression Trees)

אלגוריתם CART יודע להתמודד גם עם משתנים קטגוריים וגם עם משתנים מספריים. הוא בונה עצים בינאריים - כלומר בכל צומת יש רק שתי אפשרויות. במקום אנטרופיה, CART משתמש בממד ג'יני:

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

איך נמנעים מoverfitting?

נימנע מoverfitting באמצעות pruning, כאשר המטרה היא להגביל את מרחב ההיפותזות.

pre-pruning

מונעים מהעץ לגדול יותר מדי.

- להגביל את העומק המקסימלי.
- לקבוע מינימום נקודות לעלה - אם פיצול יקטין אותנו מתחת לזה, לא מפצלים.
- לא לפצל אם ה-IG נמוך מ- ϵ מסויים.

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-05-18

post-pruning

בונים את העץ עד הסוף - ואז מתחילים לקצוץ.
בשביל להחליט מה לקצוץ, נבדוק את העץ לפי validation data. בכל מקרה כל
קיצוץ יפגע קצת ב training data, אבל אם הפגיעה ב validation data לא גדולה אז כנראה
שהחלוקה הייתה overfitting ואפשר לוותר עליה.

Boosting

עצי החלטה הם מודל מאוד חזק, אבל יש לו חסרונות:

- הם יכולים לפצל רק לפי פיצ'ר אחד בכל פעם (כלומר הם לא יכולים לעשות חלוקה לפי פונקציה לינארית)
 - לא יציבים - הבדלים קטנים בנתונים האימון יכולים לגרום להבדלים מהותיים בעץ.
- (אפשר ללמוד עצי החלטה הרבה פעמים (בכל פעם מגבילים את המאפיינים שאפשר ללמוד, כדי לקבל עץ שונה). זה נקרא Random Forest)

Weak Learners

הגדרה: Weak Learner הוא כל מודל שעובד קצת יותר טוב מניחוש אקראי. הלומד החלש לא צריך לתת תשובה מדויקת - אבל הוא צריך להביא לנו משהו.

Boosting

Boosting שייך למשפחה של Ensemble Learning - שילובים של מודלים שונים (נקראים base models או weak learners) למודל אחד יותר חזק. התוצאה בד"כ היא משהו שיש לה bias נמוך וvariance נמוך.

(XGBoost) Gradient Boosting

יש לנו מדגם אימון מתוייג:

$$\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$

נדגם מתוך התפלגות $\mathcal{D}$ כאשר $\mathbf{x}_i \in \mathbb{R}^d$ ו $y_i \in \mathbb{R}$. פונקציית הloss שלנו היא $\ell(\text{true}, \text{predicted})$:

$$\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))$$

המטרה: לבנות מודל מדויק ע"י שילוב כמה מודלים. אנחנו רוצים ללמוד את:

$$f(x) = \sum_{m=1}^M \gamma_m h_m(x)$$

מספר העצים בensemble שלנו.	M	כאשר:
Weak Learner כלשהו.	h_m	
המשקל של העץ הספציפי.	γ_m	

הרעיון - בונים עצים, כאשר כל עץ מנסה לחזות את הטעויות של הensembles הנוכחי. כלומר:

$$f_{\text{new}}(\mathbf{x}) = f_{\text{old}}(\mathbf{x}) + h_{\text{next}}(\mathbf{x})$$

כאשר h_{next} לומד את x_i אבל במקום y_i התוויות שלו הן Err של f_{old} . כדי לפשט את האלגוריתם אפשר להתחיל עם $\hat{y}_i^{(0)} = 0$ (ואז השגיאה תהיה y_i המקוריים)

Gradient Descent במרחב הפונקציות

אלגוריתם boosting זה הוא בעצם gradient descent. עבור מרחב כל הפונקציות האפשריות, אנחנו מנסים למצוא פונקציה שממזערת את הloss שלנו. אם היינו יכולים לגזור את מרחב הפונקציות, היינו מעדכנים

$$f_{t+1} = f_t + \alpha \frac{\partial \mathcal{L}(f)}{\partial f}$$

אנחנו לא באמת יכולים לגזור לפי פונקציה, אבל בגלל שהloss מוגדר רק בנקודות האימון אנחנו יכולים לשערך את הגרדיאנט של המרחב האינסופי לפי גרדיאנט במרחב ה-n-מימדי שמוגדר לפי נקודות האימון.

בעיות מעשיות

- העץ צריך להיות רדוד יחסית, כדי שהוא לא ילמד רעש.
 - MSE לא טוב לנו, כי אז XGBoost יתקש לתקן לכיוון outliers.
- עדיף לקחת עצים חלשים - שלכל אחד מהם יש אמנם הרבה רעש, אבל החיבור שלהם יתן פרדיקציה טובה.

AdaBoost עם Adaptive Boosting

AdaBoost מנסה ללמוד גם את המקדמים:

$$H(\mathbf{x}) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(\mathbf{x}) \right)$$

כל weak learner מקבל משקל α_m בהחלטה הסופית לפי רמת הדיוק שלו.